PROJECT DOCUMENTATION — TRAVEL MEMORY DEPLOYMENT

Project GIT Hub Link :
https://github.com/rajkumaramithn/TravelMemory.git

PROJECT OVERVIEW

TravelMemory is a full-stack web application designed to store and manage travel experiences. Users can add, view, and manage travel memories with images and descriptions.

The application is deployed on AWS using a scalable architecture with load balancing, domain management, and secure HTTPS access, Along with Cloud Flare CDN and a Mongo DB backend.

1. Technology Stack

Frontend:

- React.js
- HTML, CSS, JavaScript
- Static build hosted via Node/Express

Backend:

- Node.js
- Express.js
- PM2 process manager

Database:

- MongoDB Atlas

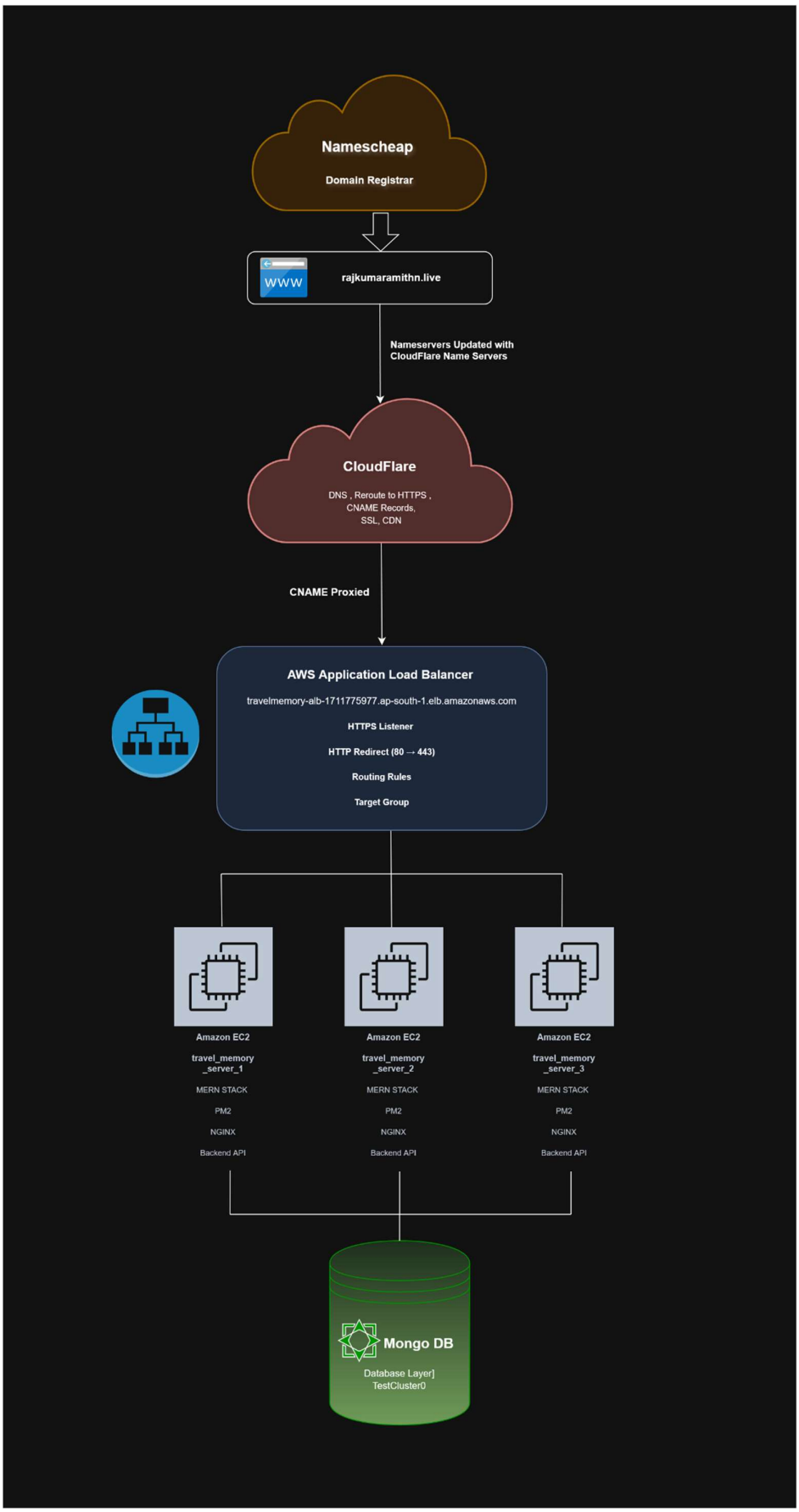
Infrastructure:

- AWS EC2 (3 instances)
- AWS Application Load Balancer (ALB)

Domain & Security:

- Namecheap (Domain registrar)
- Cloudflare (DNS + SSL + CDN)
- AWS ACM (SSL certificate)

2. Project Architecture



3. Code Deployment on EC2 Instances

In this setup we will be using 3 EC2 instances named :
travel_memory_server_1
travel_memory_server_2
travel_memory_server_3

Configuration

Cloning Repository & Adding Dependencies

sudo apt update && sudo apt upgrade -y

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-12-78:~$ sudo apt update && sudo apt upgrade -y
Get:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease [136 kB]
Get:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease [136 kB]
Get:4 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 Packages [1483 kB]
Get:5 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute/main Translation-en [524 kB]
Get:6 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64 Components [395 kB]
Get:7 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64 c-n-f Metadata [32.4 kB]
Get:8 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 Packages [16.3 MB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security InRelease [136 kB]
Get:10 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute/universe Translation-en [6329 kB]
```

i-05d12047021250cf8 (travel_memeory_server)

PublicIPs: 15.206.148.161 PrivateIPs: 172.31.12.78

sudo apt install apt nginx -y

```
ubuntu@ip-172-31-12-78:~$ sudo apt install apt nginx -y
apt is already the newest version (3.2.0).
apt set to manually installed.
Installing:
  nginx

Installing dependencies:
  nginx-common

Suggested packages:
  fcgiwrap  nginx-doc  ssl-cert

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 0
  Download size: 661 kB
  Space needed: 1874 kB / 4487 MB available

Get:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 nginx-common all 1.28.3-2ubuntu1 [36.4 kB]
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 nginx amd64 1.28.3-2ubuntu1 [625 kB]
Fetched 661 kB in 0s (19.0 MB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 90668 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.28.3-2ubuntu1_all.deb ...
Unpacking nginx-common (1.28.3-2ubuntu1) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.28.3-2ubuntu1_amd64v3.deb ...
Unpacking nginx (1.28.3-2ubuntu1) ...
Setting up nginx-common (1.28.3-2ubuntu1) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service' → '/usr/lib/systemd/system/nginx.service'.
Setting up nginx (1.28.3-2ubuntu1) ...
 * Upgrading binary nginx
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for ufw (0.36.2-9build1) ...
Scanning processes...
Scanning candidates...
Scanning linux images...
```

i-05d12047021250cf8 (travel_memeory_server)

PublicIPs: 15.206.148.161 PrivateIPs: 172.31.12.78

sudo npm install -g pm2

git clone https://github.com/rajkumaramithn/TravelMemory.git
cd TravelMemory

cd backend
npm install

Environment Configuration

nano .env

PORT=3000
MONGO_URI=mongodb+srv://rajkumaramithn_db_user:thundercat%405544@testcluster0.vpiprb5.mongodb.net/?appName=TestCluster0
JWT_SECRET=supersecretkey

nano index.js

```
  GNU nano 8.7.1
const path = require('path')
const express = require('express')
const cors = require('cors')
const os = require('os');
require('dotenv').config()
const app = express()
PORT = process.env.PORT
const conn = require('./conn')
app.use(express.json())
app.use(cors())
app.use(express.static(path.join(__dirname, '../frontend/build')))
const tripRoutes = require('./routes/trip.routes')

app.use('/trip', tripRoutes) // http://localhost:3001/trip --> POST/GET/GET by ID

app.get('/hello', (req,res)=>{
    res.send('Hello World!')
})

app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname, '../frontend/build', 'index.html'))
})

app.get('/whoami', (req, res) => {
    res.send({
        instance: os.hostname(),
        message: "Request served by this EC2 instance"
    });
});

app.use('/api/trip', tripRoutes);

app.get('*', (req, res) => {
    res.sendFile(path.join(__dirname, '../frontend/build', 'index.html'))
})

app.listen(PORT, "0.0.0.0",() => {
    console.log(`Server started at http://localhost:${PORT}`)
})
```

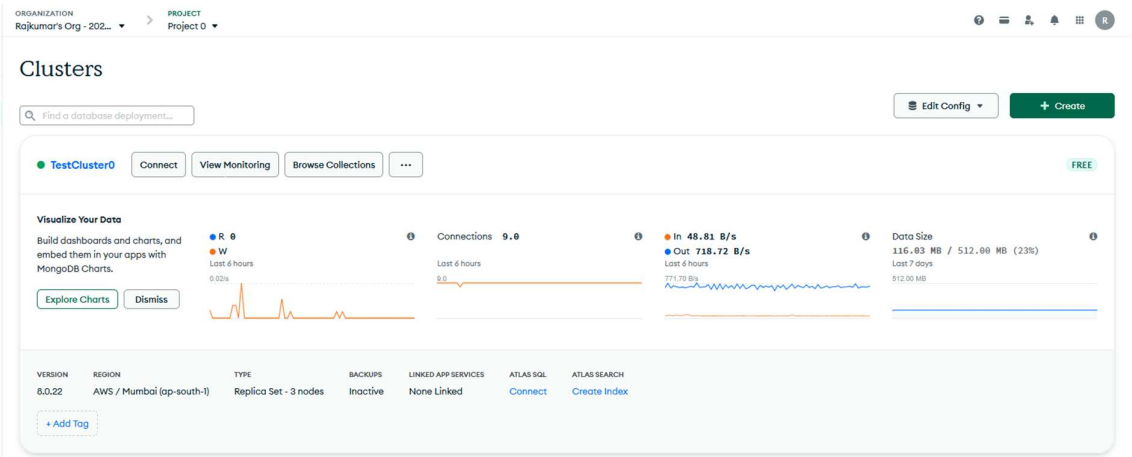
Starting Backend Using PM2

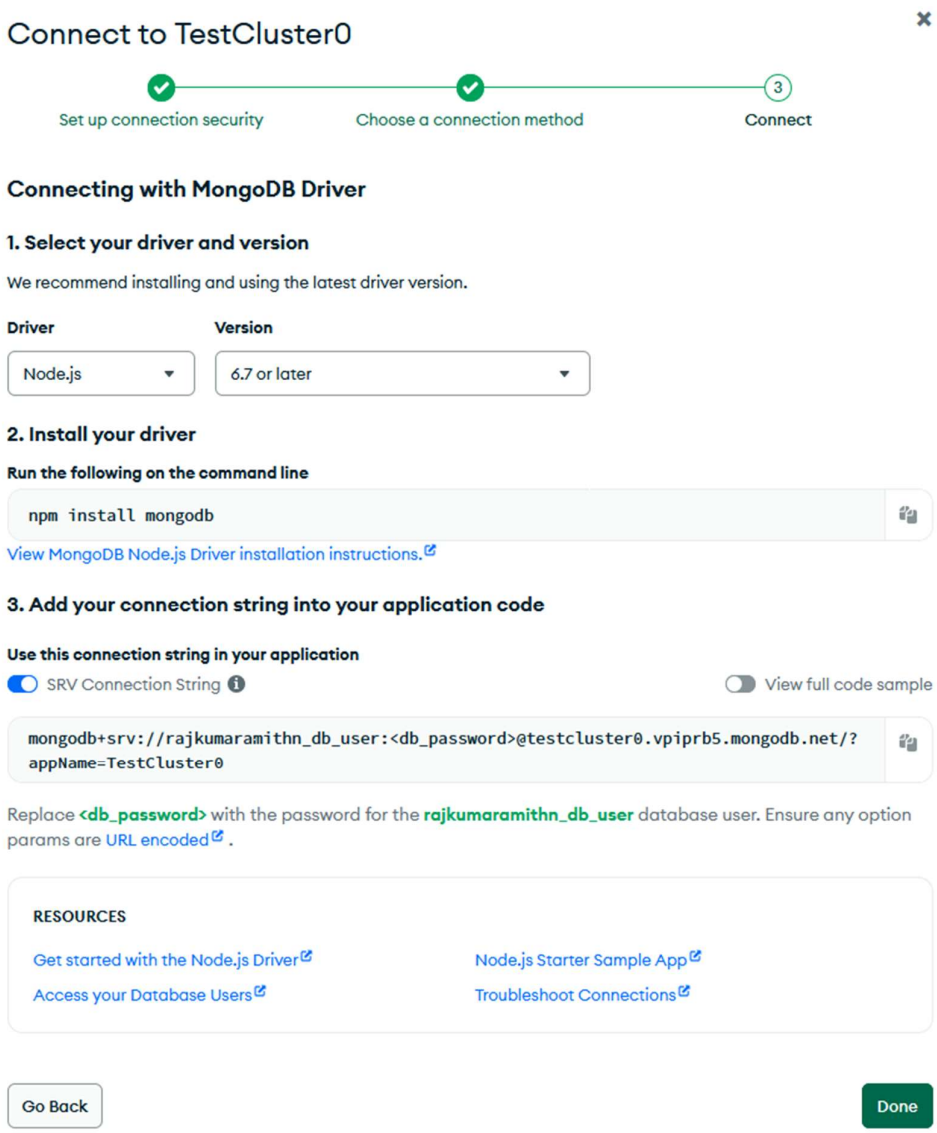
pm2 start index.js --name backend
pm2 save
pm2 startup

```
ubuntu@ip-172-31-12-78:~/TravelMemory/backend$ nano .env
ubuntu@ip-172-31-12-78:~/TravelMemory/backend$ nano .env
ubuntu@ip-172-31-12-78:~/TravelMemory/backend$ pm2 start index.js --name backend

                        -------------

__/\\\\\\\\\\\\\\____/\\\\____________/\\\\____/\\\\\\\\\_____
 _\/\\\\\\\\\\\\\\\___\/\\\\\\\\\\\\\\\
```

4. Mongo DB Cluster Connection

Create a Mongo DB Cluster for this setup and copy the Driver URI replace the username section with your Mongo DB username.
(Note: I have not Included my username in this one command alone for security reasons).





5. Frontend Setup

Installing Dependencies

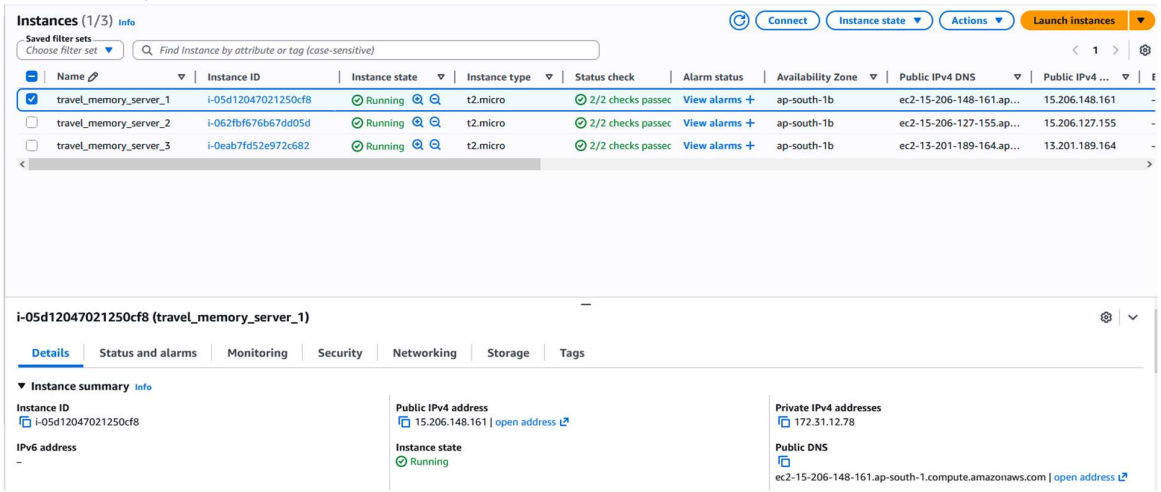
cd ../frontend
npm install

npm run build

Move to NGINX

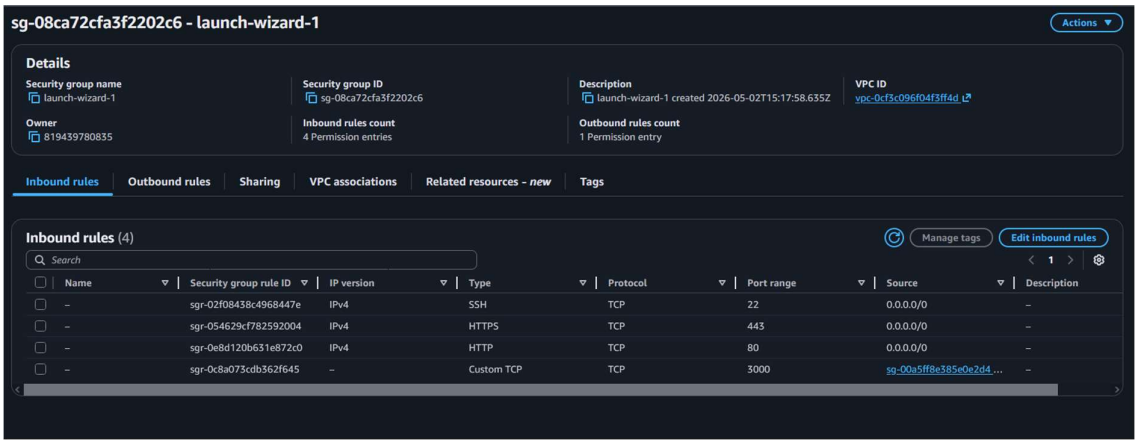
sudo rm -rf /var/www/html/*
sudo cp -r build/* /var/www/html/

Create multiple EC2 Instances



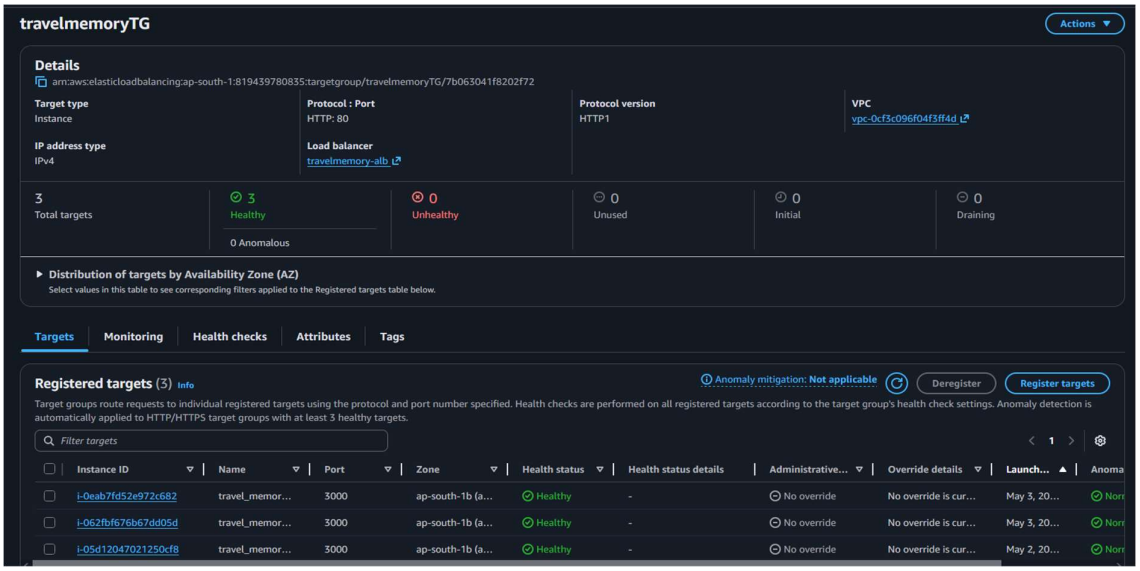
6. Security Group Settings for all EC2 Instances

protocol	Port No	Source
SSH	22	0.0.0.0/0
HTTP	80	0.0.0.0/0
HTTPS	443	0.0.0.0/0
Custom TCP	3000	ALB Security Group



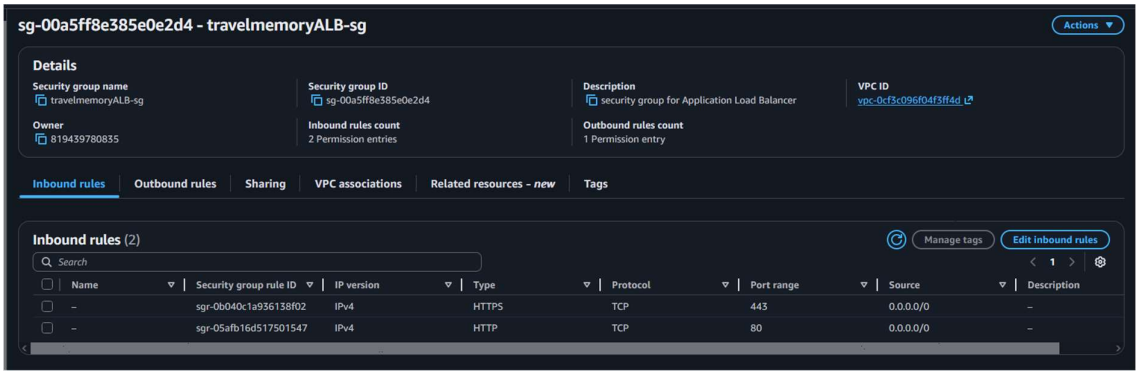
7. Load Balancer

Setting up the Target Group a prerequisite for the Load Balancer by including all three instances in to the Target Group which will should show up 3 Healthy Targets



Security Group Settings for all ALB

protocol	Port No	Source
HTTP	80	0.0.0.0/0
HTTPS	443	0.0.0.0/0



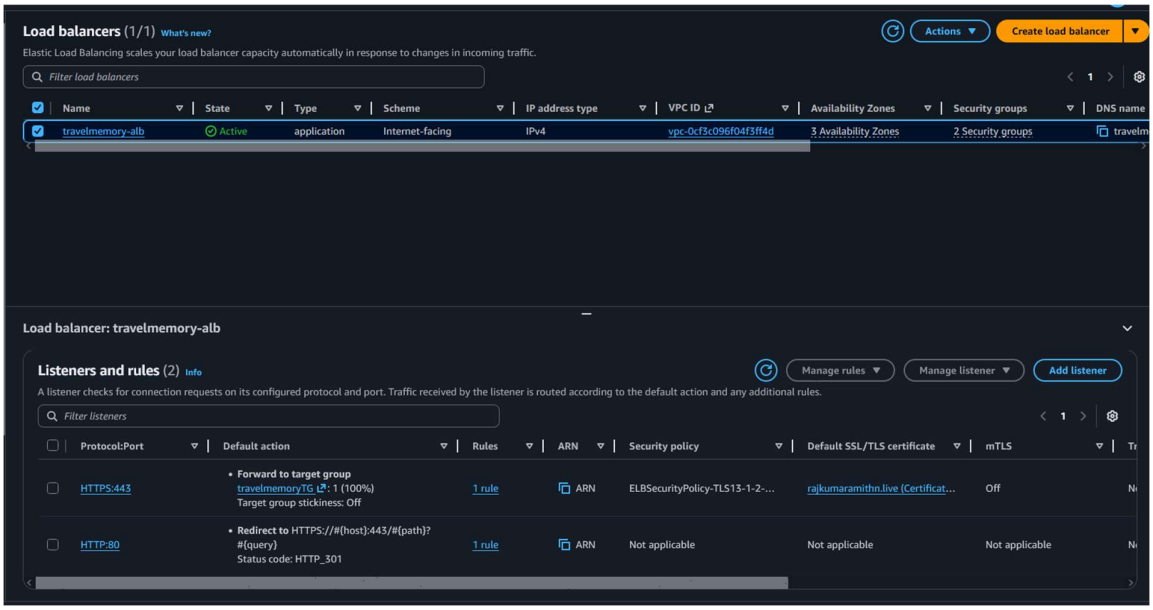
Final Load Balancer Setup

Name: travelmemory-alb

Scheme: Internet-facing

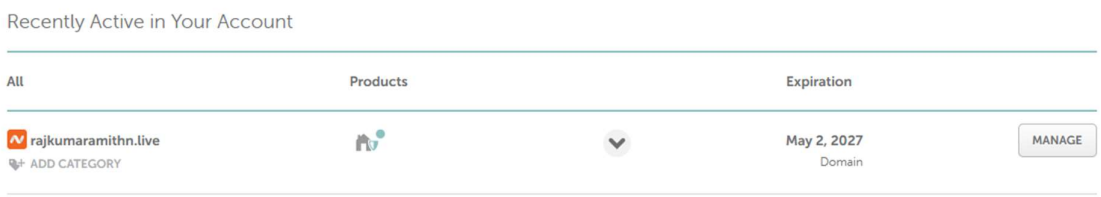
Listener: HTTP

Port No: 80



8. Acquiring a Custom Domain Name from Namescheap.com

In this case we will be using a custom domain name bought from Namescheap.com , However available custom domain name can be used for the demonstration.



9. Setting Up Cloudflare DNS

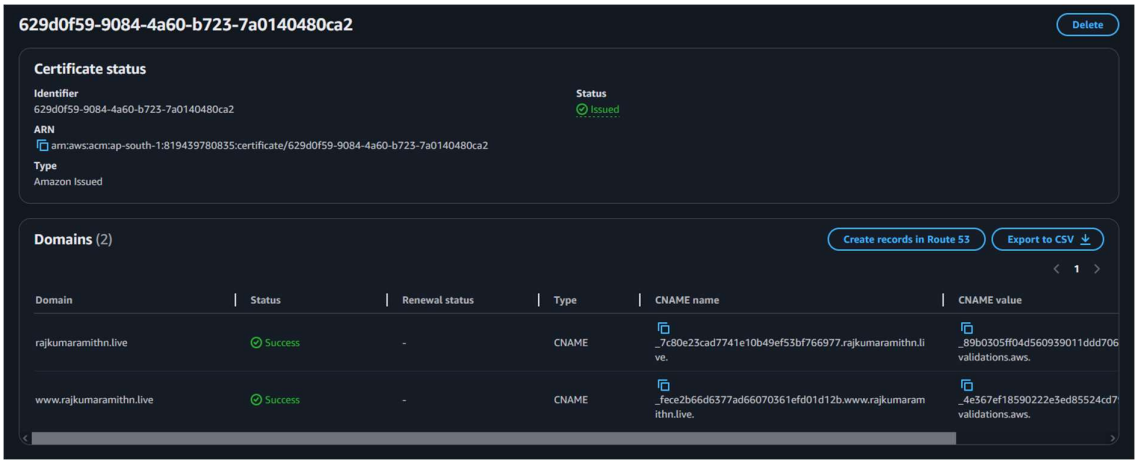
Open the DNS tab on the Cloudflare console and add the following Records

Type	Name	Content	Proxy Status
CNAME	www “or” @	rajkumaramithn.live	Yes
CNAME	rajkumaramith.live	travelmemory-alb-1711775977.ap-south-1.elb.amazonaws.com (This is the DNS of travelmemory alb)	Yes

Requesting SSL Certificate from Amazon Certificate Manager

Open ACM on AWS portal and Request for a public certificate and add both variations of the custom domain name.
rajkumaramithn.live
www.rajkumaramithn.live

This will create a certificate request and generate its own CNAME name & CNAME value

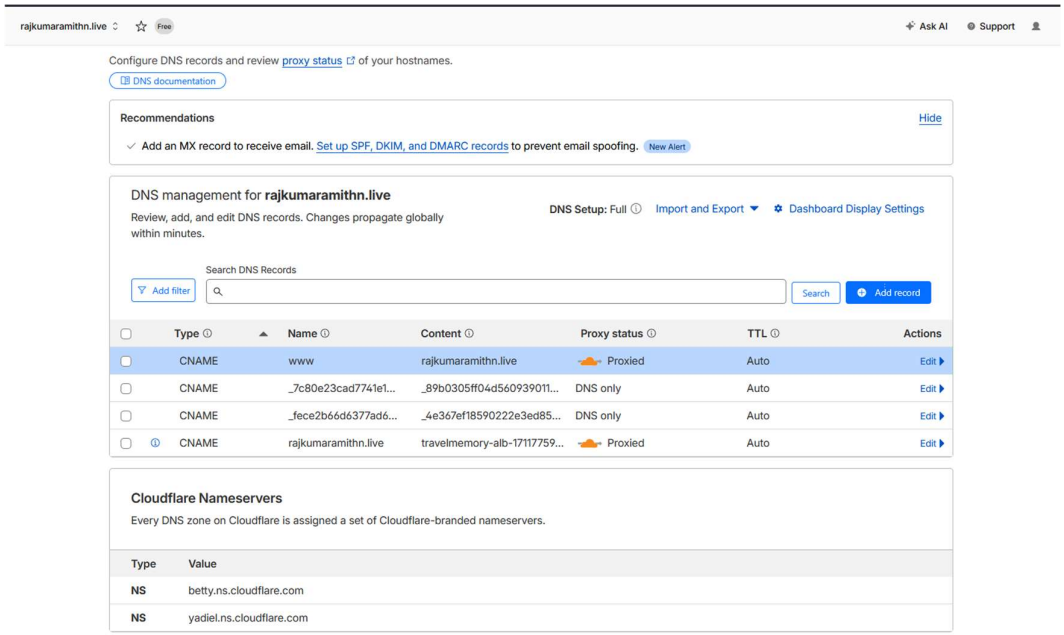


These values should be added into the DNS records of Cloudflare as CNAME values.

Type	Name	Content	Proxy Status
CNAME	_7c80e23cad7741e10b49ef53bf766977.rajkumaramithn.live.	_89b0305ff04d560939011ddd7063a892.jkddzztszm.acm-validations.aws.	No
CNAME	_fece2b66d6377ad66070361efd01d12b.www.rajkumaramithn.live.	_4e367ef18590222e3ed85524cd79f1d0.jkddzztszm.acm-validations.aws.	No

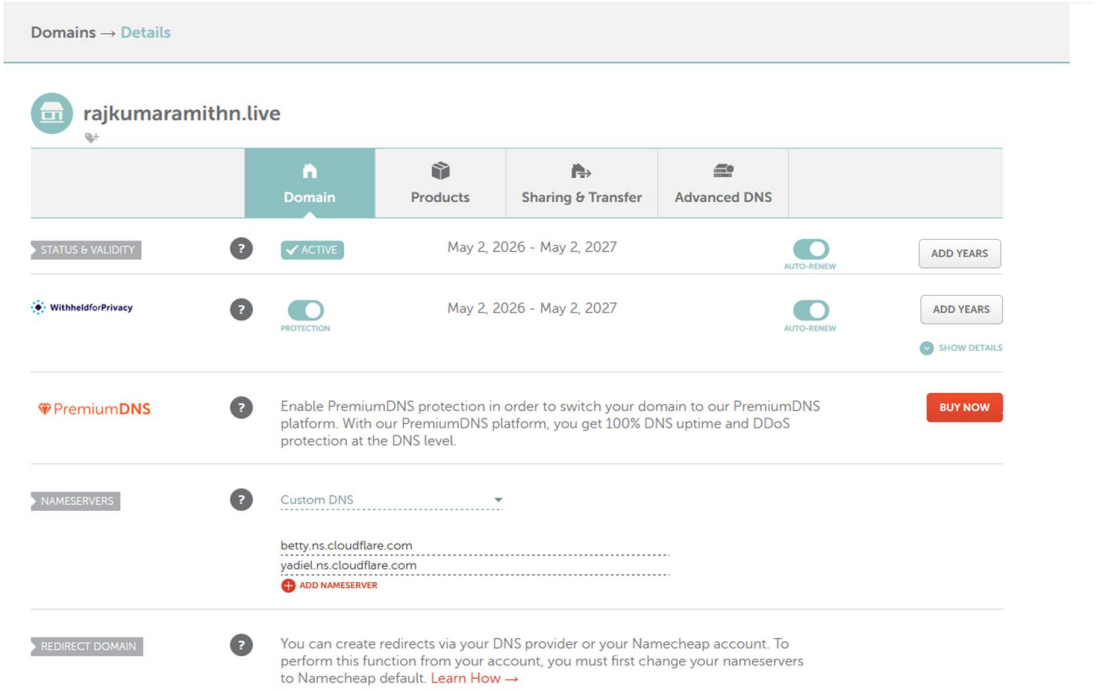
The overall Cloudflare DNS Records will be shown as below:

Type	Name	Content	Proxy Status
CNAME	www “or” @	rajkumaramithn.live	Yes
CNAME	rajkumaramith.live	travelmemory-alb-1711775977.ap-south-1.elb.amazonaws.com(This is the DNS of travelmemory alb)	Yes
CNAME	_7c80e23cad7741e10b49ef53bf766977.rajkumaramithn.live.	_89b0305ff04d560939011ddd7063a892.jkddzztszm.acm-validations.aws.	No
CNAME	_fece2b66d6377ad66070361efd01d12b.www.rajkumaramithn.live.	_4e367ef18590222e3ed85524cd79f1d0.jkddzztszm.acm-validations.aws.	No

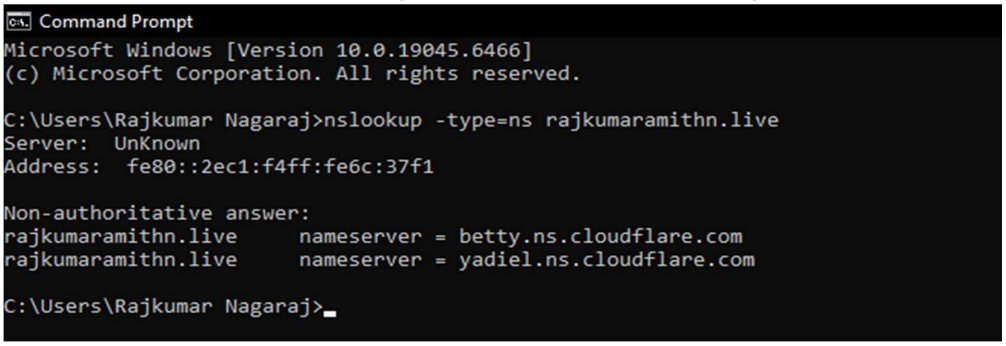


Adding the Cloudflare Nameservers onto the Domain Registrar – Namescheap

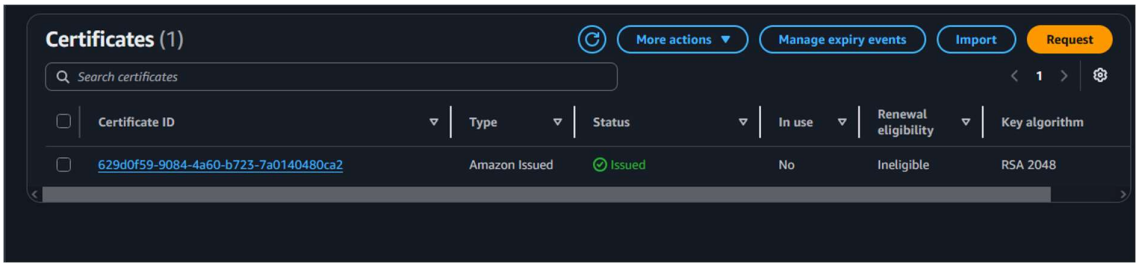
Here we will add the Cloudflare Nameservers into the Management section of the Domain Registrar – Namescheap custom DNS.



To test this on a local machine, we go to CMD and run the following commands:



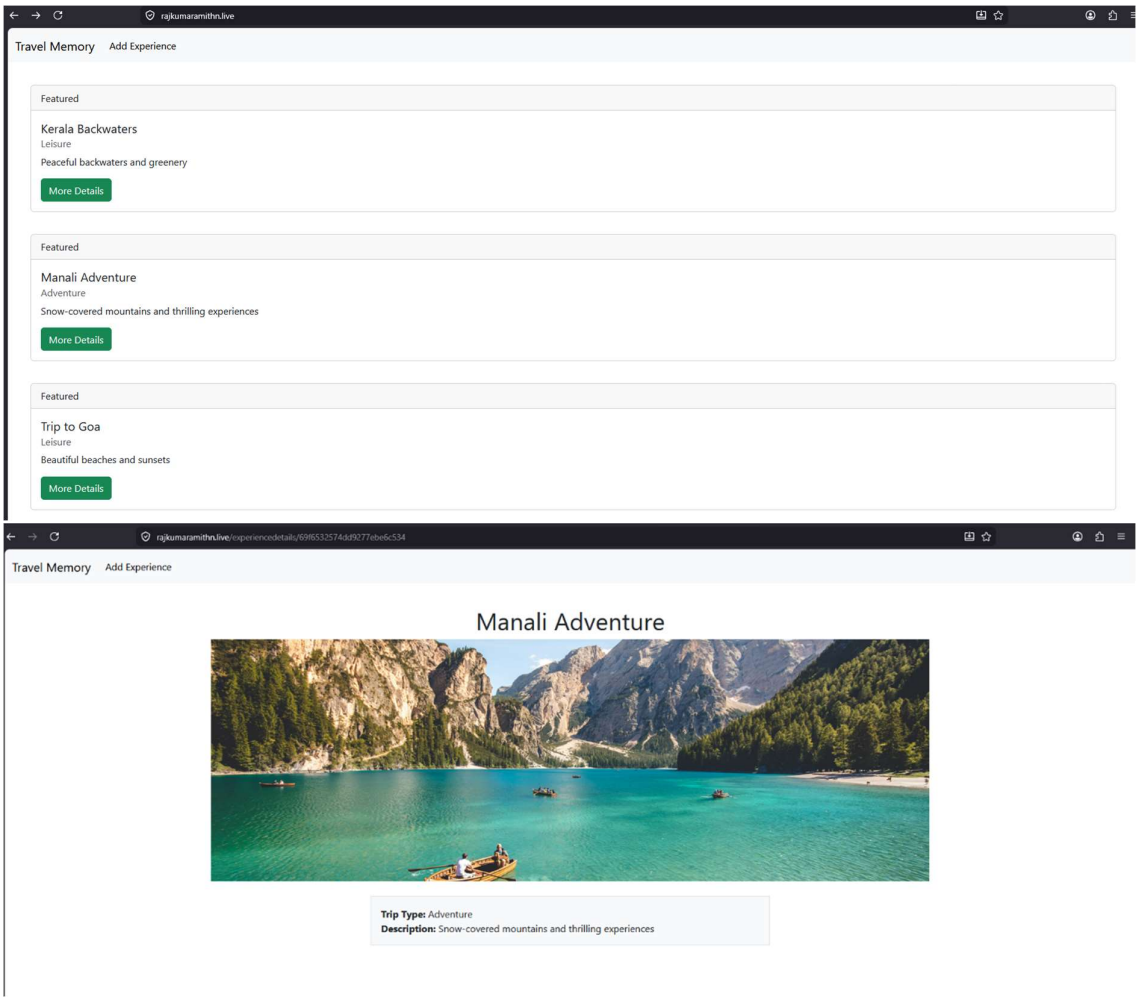
This Step will help validate the DNS mapping across Domain Registrar, Cloudflare & ACM which will make ACM issue the SSL certificate.



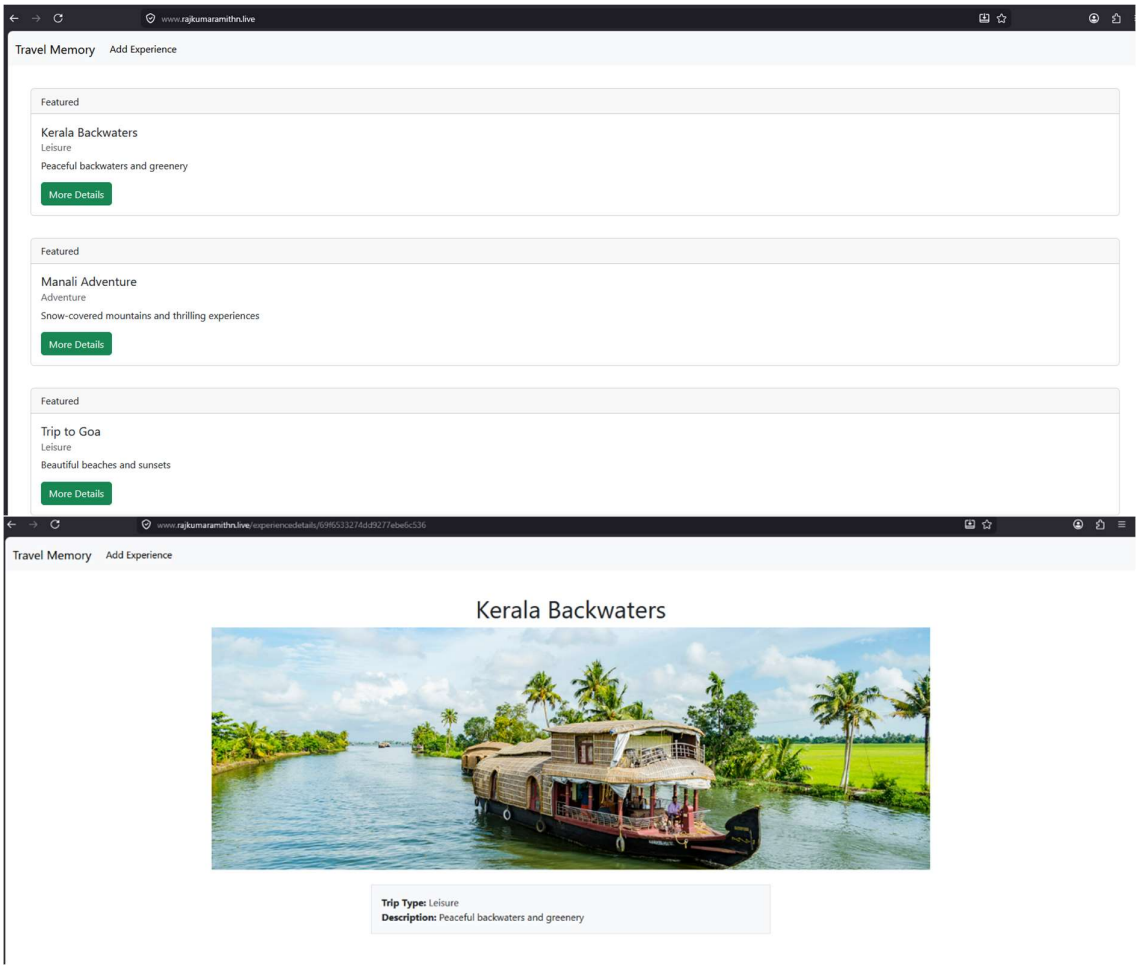
10. Custom Domain React Frontend Output

Once the setup is complete, we can use the custom domain name to view the react frontend.

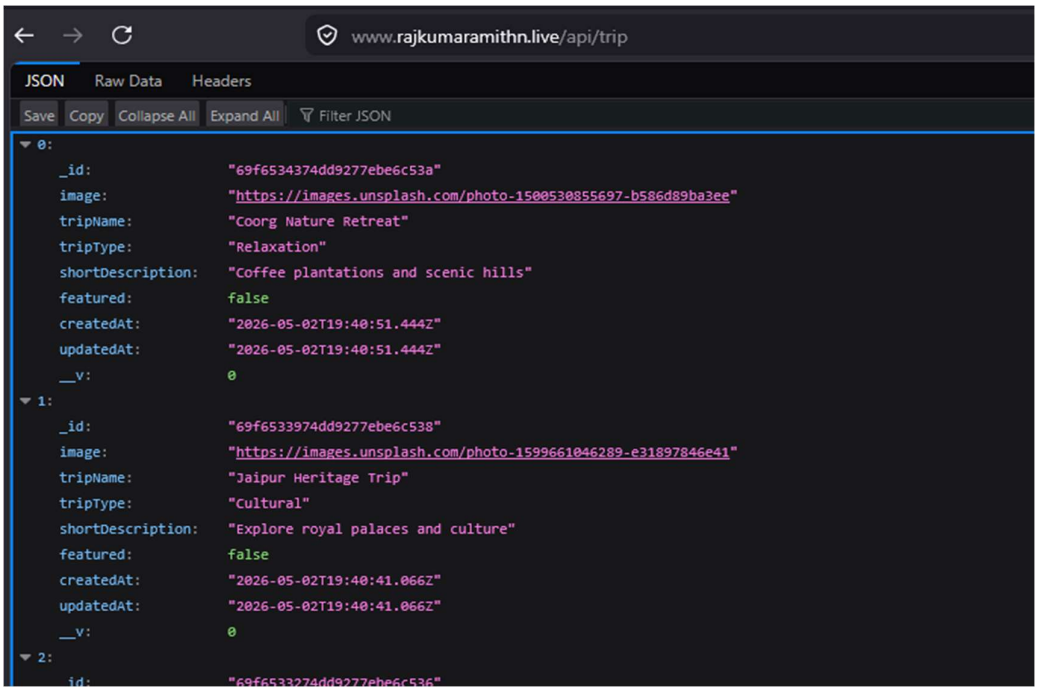
rajkumaramithn.live/



www.rajkumaramithn.live/



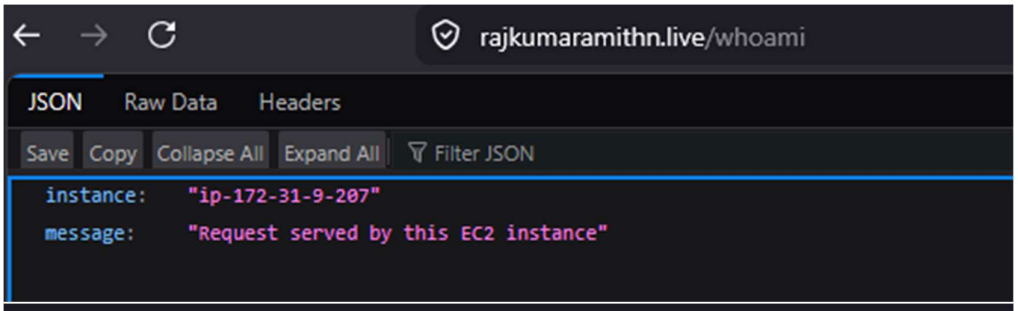
rajkumaramithn.live/api/trip **->** this shows the JSON data from the backend Mongo DB

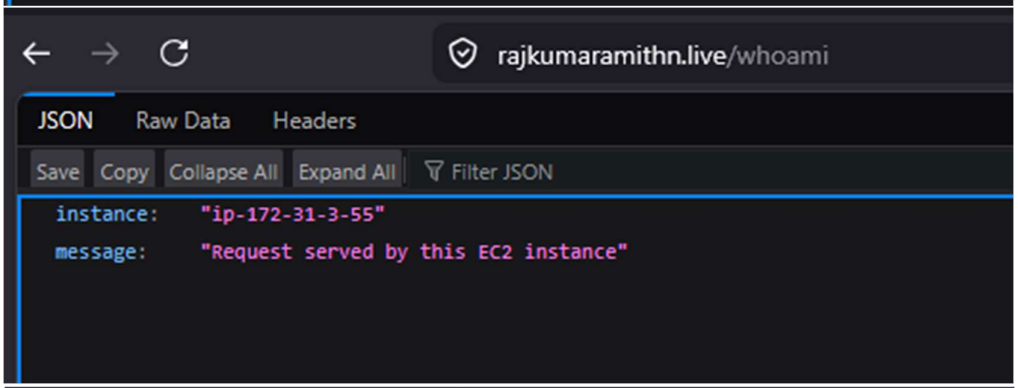


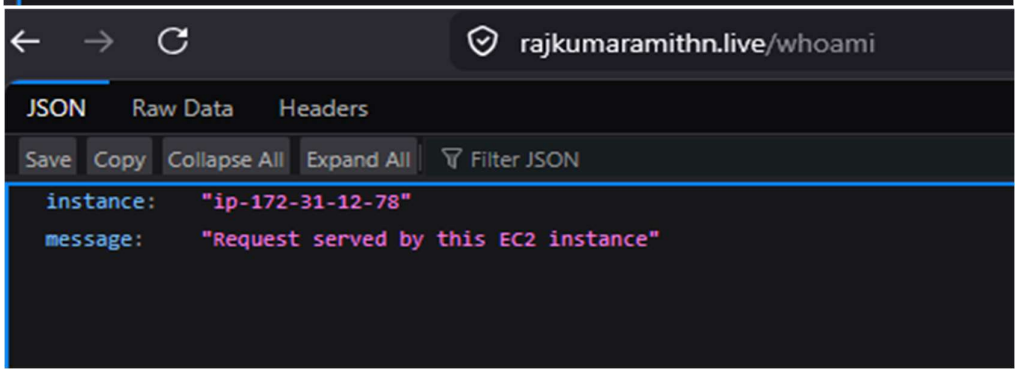
Load Balancing Demonstration

By using the flowing links and refreshing the repeatedly we can see that the load balancing is taking place successfully

https://rajkumaramithn.live/whoami







CONCLUSION

The Travel Memory application is successfully deployed using a scalable, secure, and production-grade architecture using MERN Stack. It includes load balancing, domain management, SSL encryption, and cloud-hosted backend services.

Project GIT Hub Link : https://github.com/rajkumaramithn/TravelMemory.git

Original Documentation by Rajkumar Amith Nagaraj